UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

CÁTEDRA
DISEÑO DE SISTEMAS
DE INFORMACIÓN

DonaTrack
Sistema de Gestión y Trazabilidad de Donaciones



Trabajo Práctico Anual

-2026-

V1.0

Entrega 5

- MCP, Log Centralizado y Nuevas Funcionalidades -

Objetivos de la Entrega

- **Desarrollar un componente MCP.**
- **Desarrollar las nuevas funcionalidades.**
- **Mantener un servicio de log centralizado.**
- **Extender el servicio de métricas.**

Unidades Relacionadas

- Arquitectura de Microservicios - Parte 1
- Arquitectura de Microservicios - Parte 2
- Integración asincrónica de aplicaciones
- Aplicaciones Web y Arquitectura Render Server

Consigna

MCP Server:

En esta etapa se deberá incorporar una nueva forma de interacción con DonaTrack mediante el desarrollo de un componente **MCP** y su integración con **Claude Desktop**.

El objetivo es permitir que un usuario pueda interactuar con el sistema utilizando lenguaje natural desde Claude Desktop. Para ello, Claude deberá poder consultar información y ejecutar operaciones sobre DonaTrack utilizando las herramientas (`tools`) expuestas por el servidor MCP. El componente MCP deberá ejecutarse de manera local y comunicarse con la aplicación correspondiente del alumno.

Las herramientas deberán permitir realizar las operaciones relevantes de cada componente, tanto de consulta como de operación. La definición concreta de las herramientas queda a criterio del alumno, pero deberá contemplarse el comportamiento completo necesario para poder operar el componente mediante Claude Desktop. No es necesario que cada operación disponible en la fachada se convierta automáticamente en una herramienta independiente. El alumno deberá determinar una organización razonable de las tools, procurando que sean claras, reutilizables y adecuadas para ser utilizadas por un modelo de lenguaje.

Consideraciones: El MCP Server deberá interactuar con la aplicación desarrollada por el alumno y respetar las reglas de negocio existentes en DonaTrack. No se deberán duplicar en el MCP las reglas de negocio que ya corresponden al dominio o a los servicios existentes.

Log Centralizado:

Además, se deberá incorporar un sistema de **logging centralizado** a la aplicación desarrollada.

Para ello, se provee un repositorio de ejemplo de la cátedra que contiene una implementación de referencia del sistema de logging y su integración con una aplicación. Los alumnos deberán analizar dicho ejemplo y adaptar la solución a su propio componente de DonaTrack.

https://github.com/ezequieljsosa/logs-demo-service

- En la rama principal tienen configurado un servicio online llamado Betterstack.
- Si quieren ver los logs directamente en Datadog para no tener más cuentas dando vueltas pueden usar la rama feature/datadog-logging (idea provista por el equipo Deuda técnica F.C.)

El objetivo es que los eventos relevantes generados por la aplicación puedan ser registrados y enviados a un **sistema centralizado**, permitiendo consultar los logs de los distintos componentes desde un único lugar.

La integración deberá contemplar, como mínimo:

- Incorporar el mecanismo de logging provisto en el repositorio de ejemplo.
- Configurar la aplicación para enviar sus logs al sistema centralizado.
- Registrar información suficiente para identificar el componente que generó cada evento.
- Mantener logs de los eventos relevantes de la aplicación, especialmente aquellos relacionados con las operaciones principales del dominio, que integran múltiples componentes.
- Verificar mediante una demostración que los logs generados por la aplicación pueden visualizarse correctamente en el sistema centralizado.

El repositorio de ejemplo será utilizado como referencia para la implementación. No es necesario replicar exactamente su estructura interna, pero la solución desarrollada deberá cumplir con el objetivo de centralizar los logs de la aplicación.

Sistema de métricas:

Extender el sistema de métricas con casos propios del dominio. Además, se solicita que configuren un **sistema de alarmas** para la aplicación, especialmente en métricas del negocio, que reporte el comportamiento inusual del sistema.

Funcionalidades hasta el momento:

Para esta entrega se solicita **corregir, completar y ajustar todo lo necesario** para que **todas** las funcionalidades correspondientes al *core business* de la aplicación, descritas en las entregas anteriores, queden implementadas y funcionando correctamente.

Para cumplir con este objetivo, tendrán la flexibilidad de realizar las modificaciones que consideren necesarias en los flujos, siempre que permitan completarlos y mantengan la coherencia general del sistema (es decir, pequeñas modificaciones que no sean críticas). La aplicación debe funcionar como un **todo coherente desde el punto de vista de los usuarios**, independientemente de cómo se encuentren distribuidas internamente sus funcionalidades.

Hasta el momento, las principales funcionalidades de la aplicación —es decir, aquellas que requieren la interacción de varios módulos para completarse— son:

1. **Realizar una donación.**
2. **Reportar la entrega de un paquete.**
3. **Registrar una queja sobre una donación ya entregada.**
4. **Procesar un donador (incentivos).**
5. **Registrar una nueva necesidad.**
6. **Obtener las estadísticas de un donador.**

Para poder dar cumplimiento a estas funcionalidades integradas, será necesario desarrollar, corregir o completar, dentro de cada módulo, aquellas funcionalidades internas que sean requeridas en los distintos flujos de trabajo y que, en determinados casos, funcionen como precondiciones de las funcionalidades principales. Entre ellas se encuentran, por ejemplo:

1. **ABMs de las entidades correspondientes a cada módulo.**
2. **Validaciones necesarias para garantizar la coherencia de los datos.**
3. **Configuración del matchmaking en cada depósito del módulo de Logística.**
4. Otras funcionalidades internas que sean necesarias para completar correctamente los flujos de trabajo.

Para abordar esta tarea, es altamente recomendable que, dentro de cada componente, procuren mantener el código lo más **modularizado, atómico, ordenado y comprensible** posible.

Asimismo, si lo consideran necesario, se recomienda realizar previamente una revisión del código desarrollado hasta el momento en cada módulo. Durante esta revisión, puede resultar útil identificar y dejar registrado para ustedes mismos **qué hace cada componente, qué funcionalidades se utilizan, cuáles no y qué responsabilidades tiene cada parte del código**. Esto permitirá detectar funcionalidades incompletas, código obsoleto o responsabilidades que deban ser reorganizadas.

Se recomienda prestar **especial atención al módulo de Logística**, dado que actualmente es el módulo de mayor complejidad de los cuatro y posee una mayor cantidad de interacciones y reglas de negocio.

Por último, deberán realizar **un diagrama de secuencia por cada una de las seis funcionalidades principales** mencionadas anteriormente.

Los diagramas deberán realizarse a nivel de **servicios o componentes**, mostrando principalmente:

- La interacción entre los distintos **módulos** involucrados.
- El orden en que se producen las interacciones.
- Las **principales** responsabilidades o tareas que realiza cada módulo durante el flujo (funcionalidades internas).

No es necesario representar un nivel de detalle excesivo sobre la implementación interna de cada módulo. El objetivo es poder visualizar **cómo se coordinan los diferentes componentes para llevar a cabo cada funcionalidad de negocio de principio a fin**.

Bot de Telegram:

Extender las funcionalidades del Bot de Telegram para poder hacer consultas y ABM sobre el estado de **todos** los componentes.

Entregables

1. **Documentación completa de la aplicación:** Se deberá actualizar el informe completo de la documentación de la aplicación integrada. El mismo deberá contener la siguiente información:
    a. **Modelo de dominio:** Diagrama de clases general del sistema que integre los 4 módulos.
    b. **Arquitectura:** Diagrama de despliegue actualizado del sistema integrado. Ya se va complicando.
    c. **Especificación de la API de cada componente.**
    d. **Secuencia:** Diagramas de secuencia que reflejen las funcionalidades principales hasta esta entrega.
2. **Implementación en código Java** de los requerimientos pusheado al repositorio de Github antes del horario de entrega

Evaluación

La evaluación de la entrega va a tomar un formato de “demo”. La idea es que los grupos **tengan libertad y flexibilidad para diseñar la solución como crean conveniente**, documenten el diseño y luego lo presenten y defiendan en una instancia presencial. La evaluación consta de dos partes:

Entregables:

- Deberán tener el código pusheado a los repositorios de github antes de la entrega presencial.

- Deberán subir al aula virtual el informe de documentación antes de la fecha de entrega (**03-10-2026 / 08:20hs**). El documento deberá estar en **formato PDF.**

Instancia presencial:

- Será el día **03-10-2026** en los laboratorios de la sede Medrano.
- Por una cuestión de organización, se presentarán **una cantidad determinada de grupos por rango horario**.
- La elección de horarios **se realizará con anticipación por el Discord del curso**.
- Los grupos deberán anotarse al horario que deseen presentar el trabajo el día de la entrega. Es fundamental que **sean puntuales**, es decir, que a la hora que se hayan anotado **estén listos para iniciar la presentación**, por lo que recomendamos **llegar unos minutos antes** para configurar el sistema en el laboratorio.
- La idea de la instancia presencial es que **nos cuenten qué hicieron**, presenten el sistema integrado y respondan algunas preguntas del funcionamiento del mismo.

La aprobación de la instancia es individual y la nota no la tendrán en el momento, se subirá luego a la planilla de seguimiento de alumnos.

~~Recomendaciones~~ Requisitos para la instancia presencial:

- Tengan datos necesarios previamente cargados en las bases de datos para agilizar la corrección (las **precondiciones de cada flujo**, donadores, depósito, insignias, misiones, productos, etc).
- Pueden armar colecciones en Postman para agilizar la corrección y correr todas las request en orden automáticamente.
- Para probar suele ser más sencillo que, al crear una entidad, se devuelva el ID de la entidad creada.
- Suele ser práctico tener un endpoint para borrar todos los datos de la DB, o inicializarla.
- Tenga configurado antes de arrancar la entrega un uptimerobot o similar (recordar darlo de baja después para no agotar los créditos)
- Lleve la compu con la que corre el bot de telegram, verifique que no quede prendido en ninguna máquina de la casa de nadie.